How-To Library

Informatica

Choose between available scanners and open connectivity standards to connect to data sources

© Copyright Informatica LLC 2022. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

Abstract

You can use available native resources and Informatica Advanced Scanners to scan and profile data sources.

This getting started article helps you decide which connectivity standard you need to use when you scan data sources. You can also learn more about the advantages, restrictions, and disadvantages when you use open connectivity standards such as JDBC or ODBC.

Supported Versions

- Enterprise Data Catalog 10.5

Table of Contents

Use case. 2
How to choose a connectivity mode?. 3
Using Informatica Advanced Scanners as the connectivity method. 4
Using native Enterprise Data Catalog resources as the connectivity method. 9
Using an open connectivity standard. 13

Use case

Learn why it is important to select the right connectivity mode to achieve desired business results.

A large company uses Enterprise Data Catalog to import metadata and profiling results to the catalog from the following data sources:

- Microsoft SQL Server
- Azure SQL Database
- Teradata
- PostgreSQL
- MySQL
- Non-relational MongoDB database
- Microsoft Power BI Reporting Tool
- Hadoop that uses HIVE
- Hadoop Distributed File System (HDFS)
- Databricks clusters that use Delta Tables
- An enterprise resource planning (ERP) system on SAP S/4 HANA
- Azure Data Lake Storage (ADLS) Gen2
- Microsoft Dynamics CRM
- Workday
- An application developed in-house

They use some ETL tools and stored procedures to move data from one system to another. Profiling results are particularly important for their business users.

The decision to extract metadata and profiling results from all the data sources is challenging due to the following reasons:

- Enterprise Data Catalog does not provide resources that can scan some of the data sources.
- Some native scanners do not provide profiling results.

How to choose a connectivity mode?

You can decide to use the native scanners in the Catalog Administrator tool, Informatica Advanced Scanners, or the open connectivity standards such as JDBC or ODBC to connect to data sources. Each connection method has its advantages and drawbacks.

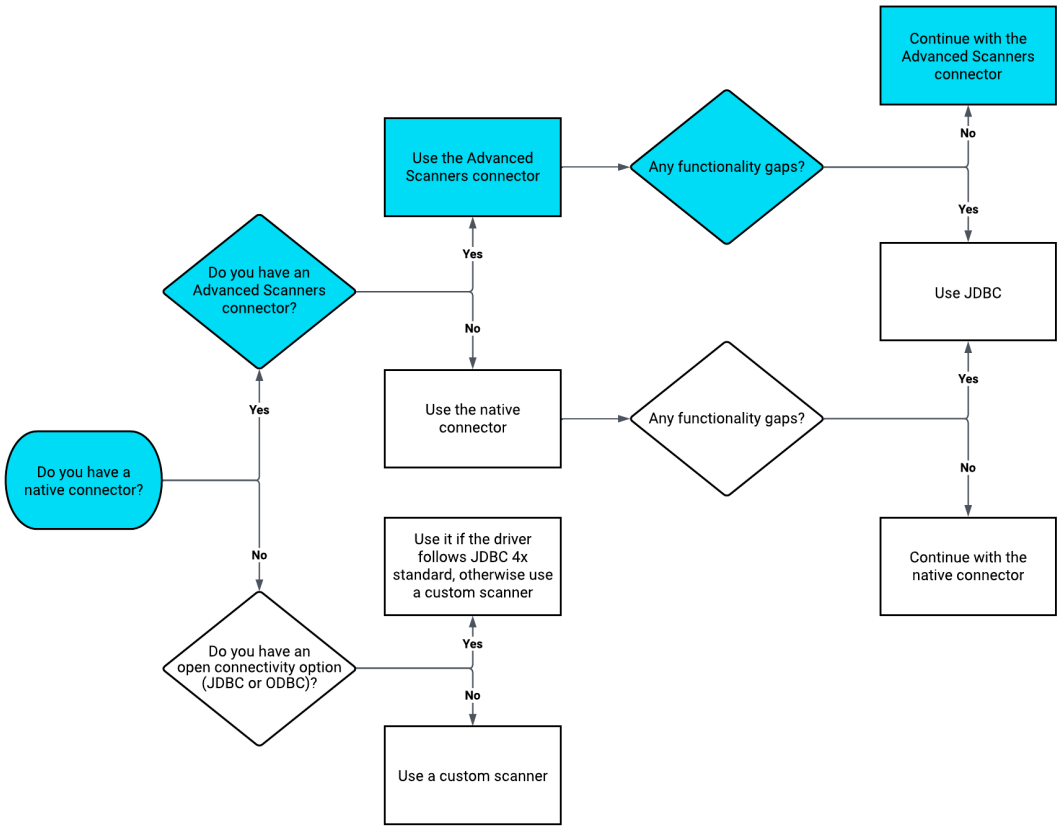
For more information about native scanners, see "Introduction to Scanners" in the *Enterprise Data Catalog Scanner Configuration Guide*.

For more information about Informatica Advanced Scanners, see "Introduction to Advanced Scanners" in the *Advanced Scanners Administrator Guide*.

Connectivity workflow

By taking an informed decision following the recommendations listed, you can achieve the desired goals faster.

The following image shows the high-level workflow that helps you decide the connectivity mode to use:



You can learn more about each mode of connecting to data sources from the following sections of this article:

- Use the Advanced Scanners connector.

  Note: Informatica Advanced Scanners is the preferred choice. You can benefit from additional parsing capabilities. For example, you can obtain lineage from stored procedures.
- Use native Enterprise Data Catalog resources.
- Use an open connectivity standard.
  You can choose a JDBC or an ODBC connection.
- Use a custom scanner.

Connectivity modes comparison

The following table compares the key decision points for connectivity options:

Decision point	Native scanners	Informatica Advanced Scanners	Open connectivity standard
Stored procedure lineage	No	Yes	Yes
PL/SQL or code parsing	No	Yes	Yes
Profiling features	Yes*	No	Yes
Ease of use	No	No	Yes
* Only some Enterprise Data Catalog native scanners support profiling features.			

For more information about the advantages and restrictions, as well as connectivity details of each method, see the remainder of this article.

Using Informatica Advanced Scanners as the connectivity method

Use Informatica Advanced Scanners whenever available.

In case when an Enterprise Data catalog native scanner is not providing features or capabilities that you need, for example, stored procedure lineage, PL/SQL, or code parsing, check whether you have access to an Advanced Scanners connector.

JDBC examples

Use the parameter descriptions and examples from this section to configure the JDBC connection.

Database definition parameters

The following table describes the database definition parameters:

Parameter	Description
url	Required. JDBC connection string.
user	Required. The name of the user.
password	Required. In plain text or encrypted with encrypt.sh or encrypt.bat utility.

Parameter	Description
driverClass	Fully qualified JDBC driver class, for example, org.netezza.Driver.

connection	Required if the target database connects to other databases. Provides mappings between the database links of the target and other MetaDex connection registry entries.
	You can use the following syntax to specify the connection:
	`connection.ID=CONNECTION_KEY=[VENDOR]:`
	`[USER@]HOST.DATABASE.[SCHEMA]`
	For example:
	`connection=MSSQL01_LINK=MSSQL-HOST.STG.DBO`
	To specify more than one link, you can append a numeric ID to the connection parameter:
	`connection.1=MSSQL01_LINK=MSSQL-HOST.STG.DBO`
	`connection.2=(HOST=abc.com)=ORACLE:ORA-H.ORA-DB.ORA-S`
	`connection.3=REGEXP:.*abc.*=ORACLE:ORA-H.ORA-DB.ORA_S`

repository.fetchStrategy	Fetches objects from the database. You can set it to one of the following values: - Default is ALL_OWNER_OBJECTS. Fetches all database objects, such as tables, views, and procedures in a particular schema to build a cache and optimize performance at the cost of increased memory consumption. - SINGLE_OBJECT. Fetches the required objects slower but with lower memory usage. Supported by Oracle and Teradata vendors.
initialSql	Specifies the SQL query executed on each connection to the database. You can use it in conjunction with initialSqlFile. The query is run every time the MetaDex server connects to the database. The query does not return any results and does not cause any side effects when executed multiple times.
initialSqlFile	Specifies the path to a file that contains SQL statements separated by a semicolon, executed when MetaDex server connects to the database. The file can be either relative to the db.properties file or absolute. You can use it in conjunction with initialSql. The query does not return any results and does not cause any side effects when executed multiple times.
catalogSchema	Overrides the default catalog tables schema.
catalogTables	Overrides the default catalog table.
vendorVersion	Stores the database version. Use only for IBM Db2 for z/OS.
db2zos.useDB2DatabaseNames	Set to true indicates that the MetaDex server uses information stored in the DBNAME column in the SYSTABLES catalog table to determine database name for a particular table. The default behavior is to use information expressed by the connection registry directory structure.

Sample JDBC connections

Examples of basic JDBC configuration:

Oracle

```
url = jdbc:oracle:thin:@oracle01.exampleorg.local:1521:cspl
user = telephone
password = telephone
vendor=ORACLE
repoType=JDBC
```

Oracle with an overridden driverClass

```
url = jdbc:oracle:thin:@oracle01.exampleorg.local:1521:cspl
user=orauser
password=orapass
vendor=ORACLE
repoType=JDBC
driverClass=oracle.jdbc.OracleDriver
```

Oracle with a single-object fetch strategy

```
url = jdbc:oracle:thin:@oracle01.exampleorg.local:1521:cspl
user=orauser
password=orapass
vendor=ORACLE
repoType=JDBC
repository.fetchStrategy=SINGLE_OBJECT
```

Oracle with specified post-connect queries

```
user=orauser
password=orapass
vendor=ORACLE
repoType=JDBC
initialSql=alter session set "_common_data_view_enabled"=TRUE;
initialSqlFile=SetupEnv.sql
```

IBM Db2

```
url = jdbc:db2://127.0.0.1:50000/SAMPLE
user = telephone
password = telephone
vendor=DB2
repoType=JDBC
```

IBM Db2 z/OS

When you extract the view definitions from a Db2 to a z/OS database, you must specify the version. Valid values are: `8`, `9`, `10` and `11`.

```
url = jdbc:db2://127.0.0.1:50000/SAMPLE
user = telephone
password = telephone
vendor=DB2ZOS
vendorVersion=10
repoType=JDBC
db2zos.useDB2DatabaseNames = true
```

Teradata

```
url = jdbc:teradata://127.0.0.1/1025/Teradata01
user = telephone
password = telephone
vendor=TERADATA
repoType=JDBC
```

Microsoft SQL Server

```
url = jdbc:sqlserver://localhost:1433;databaseName=<DatabaseName>
user = telephone
password = telephone
vendor=MSSQL
repoType=JDBC
```

Netezza

```
url = jdbc:netezza://192.168.153.128:5480/DBA
user = telephone
password = telephone
vendor=NETEZZA
repoType=JDBC
netezzaSchemasEnabled=false
```

Apache Hive

```
url=jdbc:hive2://host:port/database
repoType=JDBC
vendor=HIVE
user = user
password = pass
```

Snowflake

```
repoType=JDBC
vendor=SNOWFLAKE
url=jdbc:snowflake://account.snowflakecomputing.com?warehouse=<name>&db=<name>
user=user
password=password
```

Amazon Redshift

```
repoType=JDBC
vendor=REDSHIFT
url=jdbc:redshift://example.com:5439/database
user=user
password=password
```

Greenplum

```
repoType=JDBC
vendor=GREENPLUM
url=jdbc:postgresql://example.com:15432/database
user=user
password=password
```

SAP Hana

```
repoType=JDBC
vendor=HANA
url=jdbc:sap://example.com
user=user
password=passwordd
```

Microsoft Azure Data Warehouse (Synapse)

```
repoType=JDBC
vendor=SYNAPSE
url=jdbc:sqlserver://account.database.windows.net:1433;database=<name>;
user=user
password=password
```

Testing the JDBC connections

You can verify the JDBC connections in the MetaDex repository.

1. Open the directory where you installed the MetaDex tool.
2. From the command line, run the `connregtest` script for your operating system"

   connregtest.bat

   The following example shows a sample script:

   ```
   >connregtest.bat
   Usage:
   connregtest.bat <mdrepopath> <option>+
   Possible options:
   * connregtest.bat <mdrepopath> -list (initializes repository and lists all
   host.db combinations with their repo types, db types,
   jdbc URLs and usernames)
   * connregtest.bat <mdrepopath> -test host.db (finds host.db in mdrepo and
   verifies it)
   * connregtest.bat <mdrepopath> -test ALL (finds all db.properties in mdrepo
   and verifies them)

   >connregtest.bat d:\AdvancedScanners\mdrepo -list
   ...
   ------------------------------
   HOST.DB   REPOTYPE     VENDOR     USER     URL
   HH.BB     JDBC         TERADATA   u1       jdbc:teradata://terahost
   H2.B2     JDBC         ORACLE     u2
   jdbc:oracle:oci:@oraclehost:1521:sid

   >connregtest.bat d:\AdvancedScanners\mdrepo -test HH.BB
   ...
   HH.BB       : SUCCESS
   ```

   connregtest.sh

   The following example shows a sample script:

   ```
   connregtest.sh /home/AdvancedScanners/mdrepo -list
   ```

JDBC drivers

MetaDex provides most of the required JDBC drivers in the installation package.

The installation package contains licensed JDBC drivers for the following databases:

- Oracle
- IBM Db2
- Microsoft SQL Server
- Sybase ASE
- Informix
- MySQL
- PostreSQL
- Microsoft JDBC driver for SQL Server

If needed, download other drivers from their vendors' websites. For example, Netezza or Teradata. You can then create a jdbc folder and copy the JDBC driver files into the following location: ${INFA_HOME}/services/CatalogService/AdvancedScannersApplication/app.

Using native Enterprise Data Catalog resources as the connectivity method

Use native Enterprise Data Catalog connectors, unless you identify a functionality gap or a restriction.

For example, only some Enterprise Data Catalog native scanners, such as Oracle or Microsoft SQL Server, support profiling features. If your business needs require profiling, use an open connectivity standard to connect to a database.

If your company does not allow data staging for security compliance reasons, use an open connectivity standard to connect to connect to a database.

JDBC

You can use a JDBC connection to access tables in a database.

Objects Extracted

JDBC with MySQL

Enterprise Data Catalog extracts metadata of the following schema objects from a MySQL and AS400 databases that uses JDBC for connectivity:

- Tables
- Constraints (NOT NULL , UNIQUE , CHECK , PRIMARY KEY , FOREIGN KEY , DEFAULT)
- Clone Table
- Views
- Triggers
- Function
- Procedure
- Materialized View
- MV Trigger

JDBC with AS400

- Tables

- Constraints (NOT NULL , UNIQUE , CHECK , PRIMARY KEY , FOREIGN KEY , DEFAULT)
- Alias
- Duplicate Table
- Sequences
- Views
- Triggers
- Function
- Procedure
- User defined data types

Permissions to Configure the Resource

You must configure the CONNECT and SELECT_CATALOG_ROLE permissions for the database user account that you use to access the MySQL or AS400 databases.

Prerequisites

Download the JDBC driver JAR file from the Internet and copy the files to the `JDBCScanner` directory in the `<Informatica installation directory>/services/CatalogService/ScannerJars/externalDependencies` directory. You must restart the Catalog Service.

Alternatively, you can perform the following steps to complete the prerequisites:

1. Download the JDBC driver file and copy the file to the `<INFA_HOME>/services/CatalogService/ScannerBinaries` directory.
2. Open the `<INFA_HOME>/services/CatalogService/ScannerBinaries/CustomDeployer/scannerDeployer.xml` file and add the following lines in the file:

```
<ExecutionContextProperty isLocationProperty="true"
dependencyToUnpack="genericJDBCJars.zip"> <PropertyName>JDBCScanner_DriverLocation</
PropertyName> <PropertyValue>scanner_miti/genericJDBC/Drivers</PropertyValue> </
ExecutionContextProperty>
```

3. Save the `scannerDeployer.xml` file.
4. Recycle the Catalog Service.

Basic Information

The **General** tab includes the following basic information about the resource:

Information	Description
Name	The name of the resource.
Description	The description of the resource.
Resource type	The type of the resource.
Execute On	You can choose to execute on the default catalog server or offline.

Resource Connection Properties

The following table describes the connection properties for the JDBC resource:

Property	Description
Driver class	Name of the JDBC driver class. For more information on JDBC driver class examples, see "Examples of JDBC Driver Classes" on page 13.
URL	Connection string to connect to the database.
User	Database user name.
Password	Password for the database user name.
Agent URL	Optional. URL to the Enterprise Data Catalog agent that runs on a Microsoft Windows Server. **Note:** Make sure that you specify the URL in the following format: `http://<hostname>:<connector_port>/MIMBWebServices`

Note: To extract metadata from multiple schemas of a source using the JDBC resource type, you can specify semicolon-separated schema names when you create the resource. You can type in the multiple schema names in the **Schema** field on the **Metadata Load Settings** page.

The following table describes the **Additional** and **Advanced** properties for source metadata settings on the **Metadata Load Settings** tab:

Property	Description
Enable Source Metadata	Select to extract metadata from the data source.
Catalog	Catalog name. **Note:** You cannot use Catalog option for JDBC or ODBC sources.
Schema	Specifies a list of schemas to import.
Source Metadata Filter	You can include or exclude tables and views from the resource run. Use semicolons (;) to separate the table names and view names. For more information about the filter field, see "Source Metadata and Data Profile Filter" in the *Enterprise Data Catalog Scanner Configuration Guide*.
Case sensitivity	Select one of the following options to specify if the database is configured for case sensitivity: - Auto. If you select this option, the resource uses the JDBC API and tries to find if the database is configured for case sensitivity. If the JDBC API is not available for the resource, the resource uses the not case sensitive mode. - Case Sensitive. Select this option to specify that the database is configured for case sensitivity. - Case Insensitive. Select this option to specify that the database is configured as case insensitive.
View definition extracting SQL	Specifies the database specific SQL query to retrieve the view definition text.

Property	Description
Synonyms lineage SQL	Specifies the database specific SQL query to retrieve the synonym lineage. The following are the two columns that the query returns: - Full Synonym Name - Full Table Name
Optional Scope	Specifies the database object types to import, such as Tables and Views, Indexes, and Procedures. Specify a list of optional database object types that you want to import. The list can have zero or more database object types, which are separated by semicolons. For example, Keys and Indexes, and Stored Procedures.
Import stored procedures	Specifies the stored procedures to import. The default value is True or False whatever the case might be.
Agent Options	Specify the Enterprise Data Catalog Agent options to run the scanner job.
Memory	Specify the memory value required to run a scanner job. Specify one of the following memory values: - Low - Medium - High **Note:** For more information about the memory values, see the *Tuning Enterprise Data Catalog Performance* article.
Custom Options	Custom JVM parameters that you can set to configure scanner container. Use the following arguments to configure the parameters: - `-Dscannerloglevel=<DEBUG/INFO/ERROR>`. Changes the log level of scanner to values, such as DEBUG, ERROR, or INFO. Default value is INFO. - `-Dscanner.container.core=<No. of core>`. Increases the core for the scanner container. Verify that the value is a number. - `-Dscanner.pmem.enabled.container.memory.jvm.memory.ratio=<1.0/2.0>`. Increases the scanner container memory when `pmem` is enabled. Default value is 1. - `-DskipExternalDatabaseObjects=true`. Excludes all external objects such as schema, table, database and view. - `-DcustomSampling=<Sampling Option>`. Overrides the SQL query and limits or samples the number of rows read from the database. The **Sampling Option** of the resource is overridden with this custom option that you set for JDBC connection. See the following samples of various databases for reference: - `-DcustomSampling='fetch first 10000 ROWS ONLY'` fetches the first 10000 rows from the PostgreSQL database. - `-DcustomSampling='LIMIT 10000'` limits the number of rows read from the Vertica database to 10000. - `-DcustomSampling='TABLESAMPLE(10 PERCENT)'` samples the Databricks table to 10 percent. **Note:** This argument is applicable for column profiling and data domain discovery and there is no impact on similarity discovery.
Track Data Source Changes	View metadata source change notifications in Enterprise Data Catalog.

You can enable data discovery for a JDBC resource. For more information about enabling data discovery, see "Enable Data Discovery" in the *Catalog Administrator Guide*.

You can enable composite data domain discovery for a JDBC resource. For more information about enabling composite data domain discovery, see "Composite Data Domain Discovery" in the *Catalog Administrator Guide*.

Examples of JDBC Driver Classes

You can use a JDBC connection and a driver class to access tables in a database.

The following table provides a few examples of JDBC driver classes that belong to different data sources:

Driver Vendor	Data Source	Driver Class Example
Simba	Databricks	com.simba.spark.jdbc4.Driver
CData	MongoDB	cdata.jdbc.mongodb.MongoDBDriver
Vertica	Vertica	com.vertica.jdbc.Driver
Amazon	Athena	com.simba.athena.jdbc.Driver
Oracle	MySQL	com.informatica.jdbc.mysql.MySQLDriver
Postgres	PostgreSQL	org.postgresql.Driver
Oracle	Oracle	com.informatica.jdbc.oracle.OracleDriver
IBM	DB2	com.informatica.jdbc.db2.DB2Driver
Microsoft	SQL Server	com.informatica.jdbc.sqlserver.SQLServerDriver
SAP	Sybase	com.informatica.jdbc.sybase.SybaseDriver

Using an open connectivity standard

You must consider several factors before you choose an open connectivity standard. An open connectivity standard uses JDBC V4 or ODBC drivers to connect with your database.

- In Enterprise Data Catalog, if you want to use data profiling features, use an open connectivity driver.

  Note: Do not use JDBC V2.

  Not all Enterprise Data Catalog native scanners support profiling features. For more information about profiling, see "Suggestions for profiling warehouse" on page 14.

- Select a driver from a reliable source. The driver must come from the source system itself or from driver vendors such as CData, Simba, or DataDirect.
  The following example shows a sample JDBC driver configuration for a data source:
  `jdbc:cdata:d365sales:AuthScheme=AzureServicePrincipal;AzureTenant=<tenant>;OAuthClientId=<client>;OAuthClientSecret=<secret>;OrganizationUrl=<org URI>`

  For more information, see https://cdn.cdata.com/help/DGC/jdbc/pg_connectionj.htm.

  For more information on your driver, see applicable vendor documentation.

- The execution mode for profiling typically uses a native connector to execute profiling on the local Enterprise Data Server. Databricks tables use the Databricks mode to execute profiling on the Databricks cluster.
- An open connectivity standard provides the following benefits:
  - Open connectivity standards such as JDBC V4 or ODBC are a viable alternative to the connectors provided by Informatica.
  - JDBC is quick to set up. With ODBC, you must configure the INI files and set the right ENV variables. For example, LD_LIBRARY_PATH in Linux.

- You need not install any prerequisite libraries, except for a few data sources such as Oracle.
- Reading a data source is easy, and reading data sources is crucial to obtain profiling results.

The following example shows how to set up a JDBC driver from CData:

1. Download and install the following CData JDBC driver: cdata.jdbc.d365sales.jar.
   For more information see the CData JDBC documentation.
2. To enable profiling, ensure that you generate the license of the machine where Data Integration Service is running.
3. Copy the driver file and the license file into the following Informatica installation directory: `${INFA_HOME}/externaljdbcjars`
4. Copy driver file and the license file into the following Informatica installation directory: `${INFA_HOME}/services/CatalogService/ScannerJars/externalDependencies/JDBCScanner`
5. In the Catalog Administrator tool, restart Catalog Service and Data Integration Service.

- An open connectivity standard might cause the following challenges:
  - You can use only primitive JDBC data types. You cannot use binary or complex data types, such as arrays, maps or structs, or user-defined data types.
    For more information, see the JDBC data type documentation.
  - Specific metadata fetching. For example, you cannot fetch external tables for the Athena scanner. You need to put in a value in the developerCore.ini file.
  - Specific type of features such as incremental profiling or pushdown of sampling are not supported.
  - Execution modes. For example, Spark requires a JDBC V2 license.
    For more information, see "Suggestions for profiling warehouse" on page 14.
  - Limitations due to the driver versions provided by third party vendors.
  - JDBC third party driver limitations related to security:
    - A JDBC third party driver might only provide one method of authentication. For example, you might be allowed to enter the user and password only in plain text in the connection string.
      Your security team might not allow this type of configuration.
    - A JDBC third party driver might lack proxy support. For example, a driver might not provide any proxy support that uses Java virtual machine (JVM) options such as -Dproxy.host= -Dproxy.port

    For more information, see the *Informatica Statement of Support Related to Usage of JDBC Drivers with Informatica Enterprise Data Catalog* on the Informatica Network.

Suggestions for profiling warehouse

The profiling warehouse database stores profiling and scorecard results.

See the following recommendations to connect to the profiling warehouse:

- With native connectivity, you can use profiling warehouse with the following databases:
  - Oracle
  - IBM DB2
  - Microsoft SQL Server
  - Azure SQL Database
- With JDBC connectivity, you can use profiling warehouse with the following databases:
  - Oracle
  - IBM DB2

- Microsoft SQL Server

  You can also use profiling warehouse with MySQL, Postgres, Athena, and Vertica.

- Cloud-based databases, such as Azure ecosystem:
  - Use the Azure SQL Database with native connectivity.
  - Azure Managed SQL Server instance is fully compatible with the on-premise SQL Server and is typically used for lift and shift.
  - If the execution mode is Spark, JDBC becomes mandatory. In such cases, Azure SQL Database does not work and you must use the Azure Managed SQL Instance.
  - When the execution mode is Spark, even though the connectivity is through JDBC, Spark internally creates a JDBC V2 as it is the only supported execution mode. In such instances, you need a JDBC V2 license.

For more information about the connections that support data discovery for each resource type, see "Supported Connections" in the *Profiling Configuration Recommendations in Enterprise Data Catalog* H2L article.

For Enterprise Data Catalog profiling details, you can also see the "Profiling Data Sources" sheet in the Product Availability Matrix (PAM).

Troubleshooting third party drivers

Consider the following troubleshooting tips when you use third party drivers such as JDBC from vendors such as CData, Simba, or DataDirect:

Scanner or profiling jobs fail due to third party driver limitations

Third party driver limitations can be the reason why the scanner job logs to not provide failure details or why metadata is missing in job results.

The following scenarios might occur:

- The JDBC driver requires a license file. The file is missing from the driver location.
- Elastic Load Balancing (ELB) that you use on Amazon AWS returns a timeout error after a few minutes and terminates the connection.
- The driver does not return all metadata or some profiling is failing.

In such cases, before you contact Informatica Global Customer Support, perform the following actions:

1. Verify if the JDBC driver allows parameters for activating additional logging in the connection string.
2. Analyze the debug log of the driver to determine the root cause of your error.

Databricks Delta JDBC driver error example

```
jdbc:spark://localhost:11000;LogLevel=6;LogPath=/tmp/JDBC_logs
```

In this example, use the external query tools to verify if the JDBC connectivity works and if you can see the metadata and data properly.

CData JDBC driver error example

```
jdbc:cdata:d365sales:AuthScheme=AzureServicePrincipal;AzureTenant=<tenant>;OAuthClientId=<client>;OAuthClientSecret=<secret>;OrganizationUrl=<org URI>;Verbosity=5;logfile=/tmp/JDBC_logs/cdata_jdbc.log
```

In this example, use the external query tools to verify if the JDBC connectivity works and if you can see the metadata and data properly.

Authors

Katarzyna Strzalkowska

The author would like to acknowledge Sohan Dhanuka for help with this article.